CS 425 : DISTRIBUTED SYSTEMS - MP 3 REPORT

(GROUP-37) Akash Elumalai (akashe2) and Nikunj Agarwal (nikunja2)

a) Design: The system is implemented in Go. By default, it uses Gossip Protocol without suspicion. We use RPC for the control plane and gRPC with Protobuf for data transfer. This design allows efficient file transfers through gRPC, while keeping coordination and metadata operations lightweight via simple RPC calls.

Replication Level: We maintain a replication level of **3** to tolerate up to two node failures. In the worst case, data remains available on at least one of the replicas, and the merge and re-replication mechanism ensures it is copied to other nodes as needed.

Consistency Model: During writes, the client sends data in parallel to three servers, one primary and two replicas. A write is considered successful once an acknowledgment is received from any one server, making the system eventually consistent. If data is written to at least one server, the merge and stabilization protocols ensure that it eventually propagates to other replicas. For reads, the client first queries the primary. If it fails to respond, the system retrieves the file from a replica.

Appends, Ordering, and Merge: Each append is stored as a separate file, and the append order is maintained in metadata. During reads, we use this metadata to concatenate the append files in sequence before sending the result to the client. Each append operation includes a client timestamp and a unique AppendID(combination of filename_timestamp_clientId), and we ensure per-client append ordering during append using this timestamp. The merge process verifies data consistency by checking file checksums and appending files. If discrepancies are found, the system synchronizes the files and metadata. If metadata append ordering differs, the primary's metadata is replicated. The merge process runs periodically.

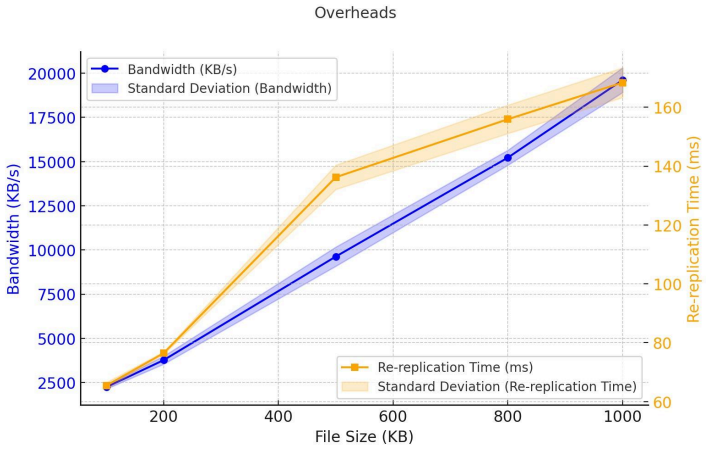
Re-Replication: Re-replication is triggered when any of the next two successors or the predecessor of a node fails. The stabilization protocol ensures that each node re-replicates the files for which it is the primary, based on the updated ring of active nodes. The node fetches metadata from its next two successors and if any file or append is missing, it reuses the existing createFile and appendFile functions to restore them. When a new node joins, some old nodes may hold redundant files outside their replication range. A background garbage collector process periodically removes such extra data.

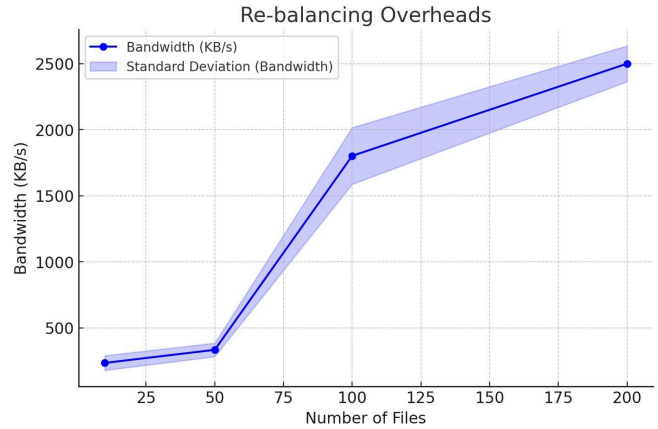
b) Past MP Use : We have extended MP2's membership protocol and the system is running with Gossip protocol without suspicion and thus trigger re-replication if that happens in my neighbour nodes. MP1 distributed grep was useful during the report compilation where we had to grep some logs which included bandwidth and time taken.

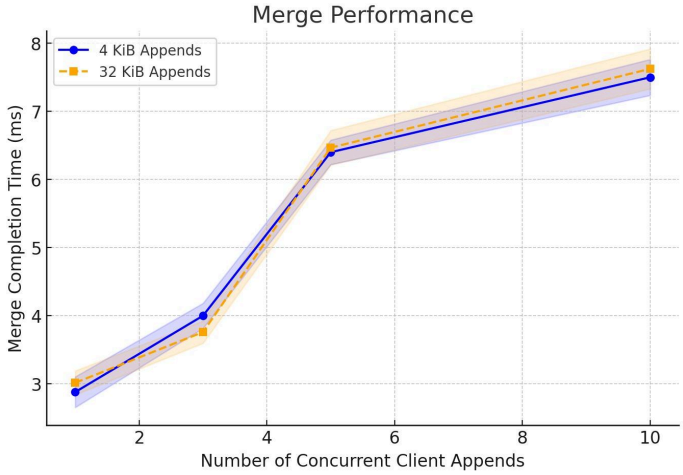
c) LLM use: We have mainly used LLMs to brainstorm design ideas and identify any flaws in our approach. It was super useful to get a better understanding of the project and to have a general conversation about the design decisions. Where it did not help was in implementing

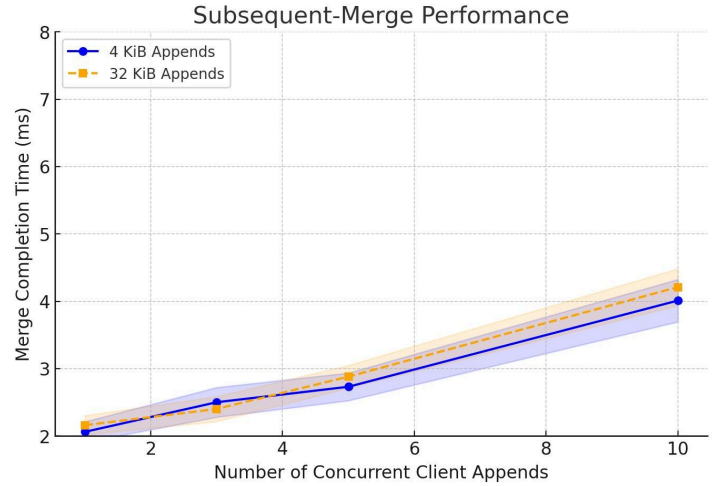
logic that required a good amount of understanding of our codebase. It struggled with maintaining context for multiple components of the project and hence, integrating its suggestions directly was not helpful. We found it most effective for clarifying concepts and boilerplate code generation, but not for actual end-to-end implementation. Refer to the “LLM usage” folder inside the hy_dfs folder for chat sessions.

d) Graphs and Measurements:









1. **Overheads** : In this experiment, we used files with sizes of 100, 200, 500, 800, and 1000 KB and failed one of the nodes(which was the same across the experiment to ensure the same files are replicated). When a node fails, re-replication is triggered across its neighboring nodes. Since the re-replication happens in parallel, we define the stabilization time as the maximum time taken among the neighbors. The bandwidth we reported is the average across the neighbors responsible for replication. The observed trend is as expected i.e. as the block size increases, the stabilization time also increases because transferring larger files takes longer. Similarly, the bandwidth increases with block size, which aligns with the fact that more data is being transferred during re-replication. On closer observation, the bandwidth is roughly 20 times the block size.

This suggests that about 20 files were transferred, and since the transfers completed within a second(which is more than the max 166 ms), we see these results.

2. **Rebalancing overheads**: In this experiment, we join one of the nodes and the reported bandwidth corresponding to the data received by this newly joined node. As expected, the bandwidth increases as more files are added. We also observe that the graph is not linear. This is expected as the distribution of files is uneven because of the hashing function, but the overall increase in Bandwidth is in line with the expectation.
3. **Merge performance:** In this experiment, we appended different files to a common file and then ran a merge operation. During the merge, since all files and appends are already present, the system only fetches and verifies the append order. If the order differs, the primary node updates the replicas with the correct order. As expected, the merge time increases slightly with more appends but the increase is modest because only metadata is being handled. For larger numbers of appends (5 or 10), the time was higher, as the primary needed to update the order on the replicas as well. We also see that the graph for 4KB and 32 KB files are almost identical. This is expected as we are dealing only with metadata and the actual file is not transferred if it already exists.
4. **Subsequent-merge performance:** During subsequent merges, the time taken didn't change much. This is again expected as the order is already sorted in the metadata and primary does not have to send the updated order back to the replicas. We also notice a small rise in the time for 10 appends. This can be because we had to get more metadata from replicas. The overall subsequent merge time was also less than the first merge which is again expected because the primary didn't have to update replica metadata orders.